FinGenieAI–Intelligent DigitalBanking & Wealth Management Platform

PROJECT NAME : FinGenieAI–Intelligent Digital Banking & Wealth Management Platform

PROJECT TYPE : Full Stack Digital Banking and Wealth Management Application

FRONTEND : ReactJS, JSX, Axios, React Router

BACKEND : Spring Boot REST APIs, Spring Security, JWT,JPA-Repository Pattern

MAJOR ROLES : Customer/User and Admin

SUBMITTED BY : Ms. KAVIYASRI M

CHAPTER 1: PROJECT INTRODUCTION

1.1 Overview

FinGenie AI is a comprehensive, enterprise-grade, full-stack digital banking and wealth management platform that combines the power of modern web technologies with artificial intelligence capabilities. The platform is built using Java Spring Boot for the backend REST API services and ReactJS for the dynamic frontend user interface.

The name "FinGenie AI" represents Financial Genie - a smart assistant that magically handles all your financial needs. Just like a genie grants wishes, FinGenie AI fulfills all banking and financial management requirements of its users through intelligent automation and AI-driven recommendations.

This platform goes beyond traditional banking applications by incorporating cutting-edge features such as real-time fraud detection using the Strategy Design Pattern, AI-powered investment recommendations, intelligent loan eligibility prediction, multi-factor authentication using OTP, and a comprehensive admin analytics dashboard.

1.2 Motivation

In today's digital era, banking has evolved from physical branches to mobile applications. However, many existing banking applications lack intelligent features that can truly assist users in managing their finances. The motivation behind FinGenie AI is to bridge this gap by creating a platform that not only performs standard banking operations but also acts as a personal financial advisor.

1.3 Project Scope

The scope of FinGenie AI covers:

- Complete user authentication and authorization system with JWT and OTP
- Full banking operations including deposit, withdrawal, and fund transfer

- Real-time fraud detection and prevention system
- AI-driven investment advisory and portfolio management
- Intelligent loan management with eligibility prediction
- nteractive AI chatbot for customer support
- Comprehensive admin dashboard for system monitoring and analytics

1.4 Objectives

The primary objectives of this project are:

1. To develop a secure full-stack banking application using Spring Boot and ReactJS
2. To implement industry-standard security practices including JWT authentication and MFA
3. To incorporate AI and machine learning concepts for fraud detection and loan prediction
4. To demonstrate the practical application of software design patterns
5. To build a scalable and maintainable application following layered architecture principles
6. To provide a seamless user experience through an intuitive and responsive UI

CHAPTER 2: PROBLEM STATEMENT

2.1 Existing System Problems

Traditional banking applications suffer from several critical limitations that impact user experience and security:

Security Limitations:

Traditional systems rely on simple username and password authentication which is vulnerable to brute force attacks, phishing, and credential theft. There is no second layer of verification to protect user accounts.

Lack of AI Integration:

Existing systems do not leverage artificial intelligence to provide personalized financial advice, detect unusual transaction patterns, or predict loan eligibility based on user financial profiles.

Limited Fraud Detection:

Most basic banking applications lack real-time fraud detection capabilities. By the time fraud is detected, significant financial damage has already occurred.

No Personalized Investment Guidance:

Traditional banking apps do not provide personalized investment recommendations based on user risk profiles, financial goals, or market conditions.

Poor Admin Analytics:

Bank administrators lack comprehensive real-time dashboards to monitor system health, customer activities, fraud patterns, and transaction analytics.

Fragmented User Experience:

Users need to visit multiple platforms for banking, investment, loans, and financial advice creating a fragmented and inefficient experience.

2.2 Proposed Solution

FinGenie AI addresses all these problems through a unified intelligent platform that provides:

1. Multi-layer security with JWT tokens and OTP-based two-factor authentication
2. Real-time fraud detection using advanced algorithmic strategies
3. AI-powered investment recommendations and portfolio analytics
4. Intelligent loan eligibility prediction with credit score analysis

5. Comprehensive admin analytics dashboard with real-time monitoring

6. Unified platform for all banking and financial management needs

7. Interactive AI chatbot for instant customer support

CHAPTER 3: TECHNOLOGY STACK

3.1 Frontend Technologies

ReactJS 18.x:

React is a JavaScript library developed by Facebook for building user interfaces. It uses a component-based architecture where the UI is broken down into reusable components. React's virtual DOM ensures efficient rendering and excellent performance. In FinGenie AI, React is used to build all user interface components including the login page, dashboard, banking operations, investment panel, loan management, fraud alerts, and admin dashboard.

Redux:

Redux is a predictable state management library for JavaScript applications. It provides a centralized store for all application state making state management across components predictable and debuggable. In FinGenie AI, Redux manages authentication state, banking data, and user information across the entire application.

Axios:

Axios is a promise-based HTTP client for making API requests. It provides an easy-to-use interface for making GET, POST, PUT, and DELETE requests to the Spring Boot backend. Axios interceptors are used to automatically attach JWT tokens to every outgoing request.

React Router DOM:

React Router is the standard routing library for React applications. It enables navigation between different pages and components without full page reloads. FinGenie AI uses React Router for

navigating between login, dashboard, banking operations, investments, loans, fraud alerts, and admin panel.

CSS3:

Custom CSS3 styling is used throughout the application to create a professional and visually appealing user interface with gradient backgrounds, card layouts, responsive design, and smooth animations.

3.2 Backend Technologies

Java 17:

Java 17 is a Long Term Support version of the Java programming language. It provides modern language features, improved performance, and enhanced security. All backend services in FinGenie AI are written in Java 17.

Sping Boot 3.x:

Spring Boot is an opinionated framework that simplifies the development of production-ready Spring applications. It provides auto-configuration, embedded servers, and starter dependencies. Spring Boot forms the backbone of the FinGenie AI backend, providing the REST API infrastructure, dependency injection, and application configuration.

Spring Security:

Spring Security is a powerful and highly customizable authentication and access-control framework for Java applications. In FinGenie AI, Spring Security is configured to use JWT-based stateless authentication, BCrypt password encoding, CORS configuration, and role-based access control.

JWT - JSON Web Token:

JWT is an open standard for securely transmitting information between parties as a JSON object. In FinGenie AI, JWT tokens are generated upon successful OTP verification and are attached to every subsequent API request in the Authorization header.

Spring Data JPA:

Spring Data JPA is a part of the Spring Data project that provides easy integration with JPA-based data access layers. It eliminates boilerplate code for database operations by providing repository interfaces with built-in CRUD operations and query generation.

Hibernate:

Hibernate is an Object-Relational Mapping framework that maps Java objects to database tables. It handles all SQL generation, database connections, and transaction management automatically.

Lombok:

Lombok is a Java library that automatically generates boilerplate code such as getters, setters, constructors, toString, and equals methods using annotations. This significantly reduces code verbosity and improves readability.

3.3 Database

MySQL:

MySQL is a widely used open-source relational database management system. FinGenie AI uses MySQL as its primary database for storing user data, account information, transactions, loan applications, and investment records.

3.4 Tools and Utilities

Maven:

Maven is a build automation tool used for Java projects. It manages project dependencies, build lifecycle, and project structure.

Postman:

Postman is an API testing tool used for testing all REST API endpoints during development.

VS Code:

Visual Studio Code is the integrated development environment used for developing the ReactJS frontend.

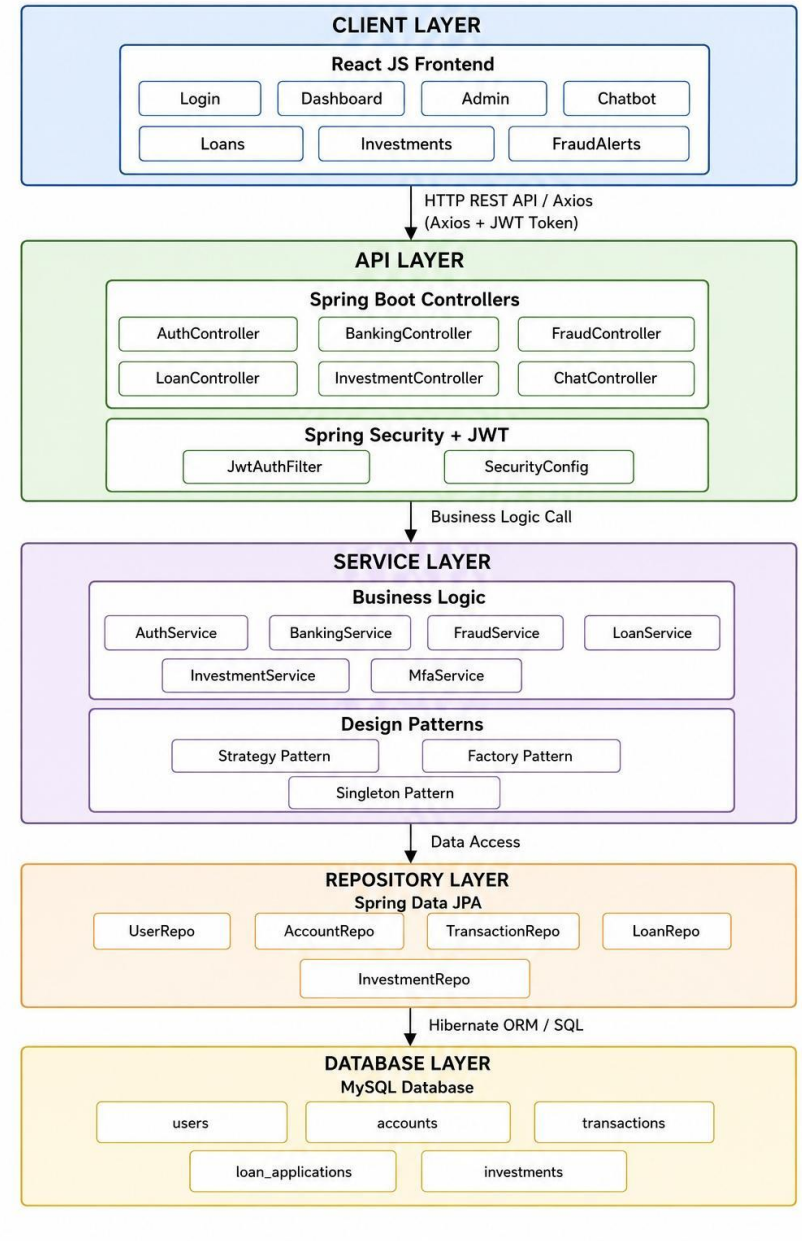
- Spring Tool Suite / IntelliJ IDEA:
- Used for developing the Spring Boot backend application.
-

CHAPTER 4: SYSTEM ARCHITECTURE

4.1 Architectural Overview

FinGenie AI follows a classic 4-layered architecture pattern that ensures separation of concerns, maintainability, scalability, and testability. Each layer has a specific responsibility and communicates only with adjacent layers.

ARCHITECTURE DIAGRAM



4.2 Layer 1 - Client Layer (Presentation Layer)

The client layer consists of the ReactJS frontend application running in the user's browser. This layer is responsible for:

- Rendering the user interface components
- Handling user interactions and events
- Managing client-side state using Redux

- Making HTTP API calls to the backend using Axios
- Implementing client-side routing using React Router
- Displaying data received from the backend

Components:

- Login and Registration pages
- Customer Dashboard with banking operations
- Investment management panel
- Loan management and EMI calculator
- Fraud alerts display
- AI Chatbot interface
- Admin analytics dashboarD

4.3 Layer 2 - API Layer (Controller Layer)

The API layer consists of Spring Boot REST controllers that expose HTTP endpoints for the frontend to consume. This layer is responsible for:

- Receiving HTTP requests from the frontend
- Validating request data using annotations
- Delegating business logic to the service layer
- Returning appropriate HTTP responses with proper status codes
- Handling cross-origin requests through CORS configuration
- JWT token validation through Spring Security filter chain

Controllers:

- AuthController - handles authentication endpoints
- BankingController - handles banking operation endpoints
- FraudController - handles fraud detection endpoints
- LoanController - handles loan management endpoints
- InvestmentController - handles investment endpoints
- ChatController - handles chatbot endpoints
- AdminController - handles admin dashboard endpoints
- MfaController - handles OTP generation and verification

4.4 Layer 3 - Service Layer (Business Logic Layer)

- The service layer contains all the business logic of the application. This layer is responsible for:
- Implementing core business rules and workflows

- Orchestrating multiple repository calls
- Applying design patterns for complex algorithms
- Handling business-level exceptions
- Performing calculations and data transformations
- Integrating AI and rule-based algorithms

Services:

- AuthService - user registration, login, password management
- BankingService - deposit, withdrawal, transfer, account management
- FraudService - fraud detection using strategy pattern
- LoanService - loan application, EMI calculation, approval prediction
- InvestmentService - investment management, portfolio analytics
- MfaService - OTP generation and verification
- ChatService - chatbot response generation
- 4.5 Layer 4 - Repository Layer (Data Access Layer)

The repository layer handles all database operations using Spring Data JPA. This layer is responsible for:

→ Performing CRUD operations on database entities
→ Executing custom queries using JPQL
→ Managing database transactions
→ Mapping Java objects to database tables through Hibernate

Repositories:

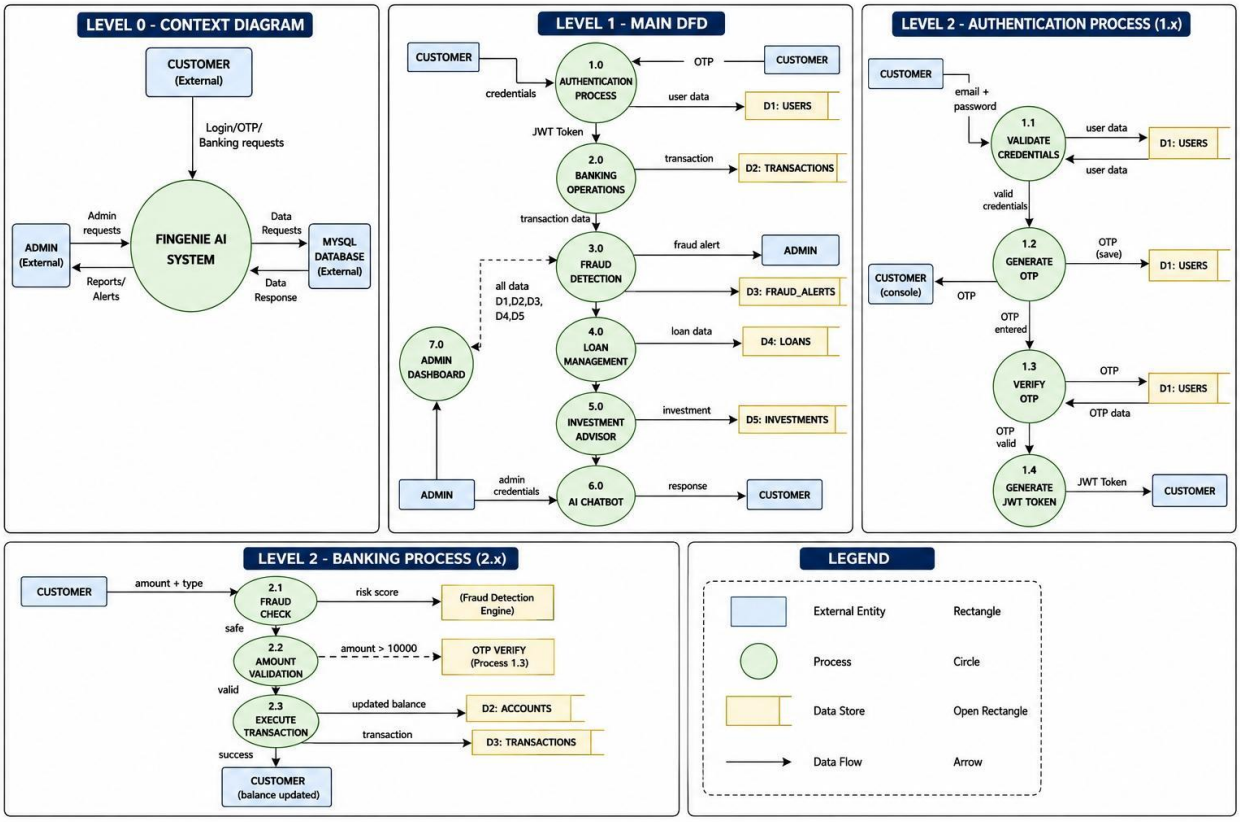
- UserRepository - user data access operations
- AccountRepository - account data access operations
- TransactionRepository - transaction data access operations
- LoanRepository - loan application data access operations
- InvestmentRepository - investment data access operations

4.6 Database Layer

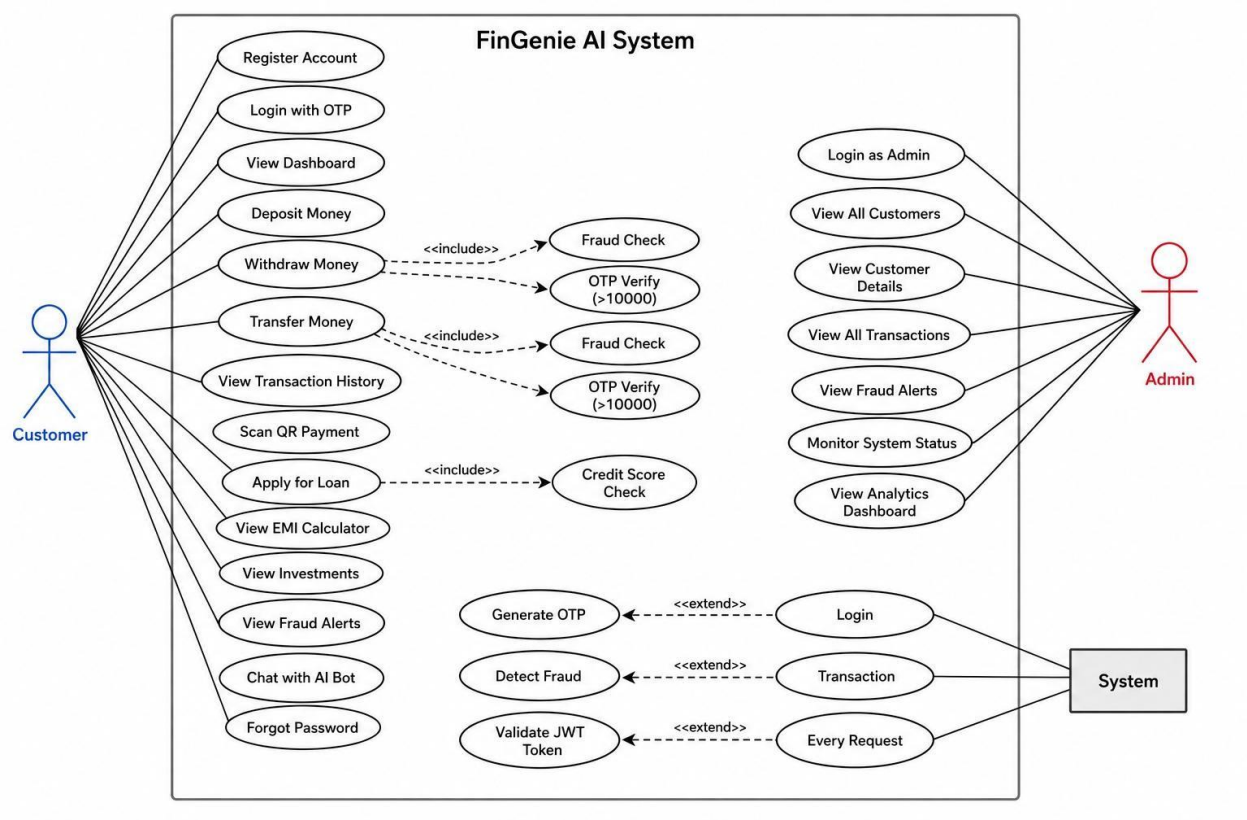
The MySQL database stores all persistent data for the application including user accounts, banking transactions, loan applications, and investment records

CHAPTER 5: DETAILED MODULE DESCRIPTION

DFD DIAGRAM



USE CASE DIAGRAM



5.1 Module 1 - Authentication and Role Management

5.1.1 Overview

The authentication module is the gateway to the entire FinGenie AI platform. It implements a robust multi-layer security system that combines JWT-based stateless authentication with OTP-based Two Factor Authentication to ensure maximum security for user accounts.

5.1.2 User Registration Process

When a new user registers on the platform:

→ The user fills in their full name, email address, and password on the registration page
→ The frontend sends a POST request to /api/auth/register with the user data
→ AuthController receives the request and delegates to AuthService
→ AuthService validates the email is not already registered
→ The password is encrypted using BCrypt with strength factor 10
→ A new User object is created with role CUSTOMER
→ The user is saved to the MySQL database
→ A JWT token is generated and returned to the frontend

→ The user is automatically logged in and redirected to the dashboard

5.1.3 Login with OTP Process

The login process involves two steps for enhanced security:

❖ Step 1 - Credential Verification:

✓ User enters email and password on the login page
✓ Frontend sends POST request to /api/auth/login
✓ AuthService retrieves user from database by email
✓ BCrypt password matching is performed
✓ If credentials are valid, a 6-digit OTP is generated
✓ OTP is saved to the user record in the database
✓ OTP is printed in the backend console for testing
✓ Response is returned with null token but user email and role

❖ Step 2 - OTP Verification:

User enters the OTP from the backend console

✓ Frontend sends POST request to /api/auth/verify-otp
✓ MfaService retrieves the stored OTP from database
✓ OTP matching is performed
✓ If valid, OTP is cleared from database
✓ JWT token is generated with user email and role claims
✓ Token is returned to frontend
✓ Frontend stores token in localStorage
✓ Role-based redirect performed - ADMIN to /admin, CUSTOMER to /dashboard

5.1.4 JWT Token Structure

→ The JWT token contains:
→ Subject - user email address
→ Role claim - ADMIN or CUSTOMER
→ Issued at - token creation timestamp
→ Expiration - token validity period
→ Signature - HMAC SHA256 signature for verification

5.1.5 Role Based Access Control

→ FinGenie AI implements two distinct user roles:
→ CUSTOMER Role:
→ Access to personal banking dashboard
→ Deposit, withdraw, and transfer operations
→ View personal transaction history

→ Apply for loans and view loan status
→ View and manage investments
→ View personal fraud alerts
→ Access AI chatbot
→ ADMIN Role:
→ Access to admin analytics dashboard
→ View all customers and their details
→ Monitor all transactions across the system
→ View system-wide fraud alerts
→ Access system health metrics
→ View total balances and transaction statistics

5.2 Module 2 - Smart Banking Operations

5.2.1 Overview

The banking operations module provides all core banking functionalities that a customer needs for their day-to-day financial transactions. It implements robust business rules including fraud detection integration and OTP verification for high-value transactions.

→ **5.2.2 Account Management**
→ When a customer logs in for the first time, the system automatically creates a bank account:
→ A unique account number is generated with FG prefix
→ Initial balance is set to zero
→ Account is linked to the user in the database
→ Account number is displayed on the dashboard

5.2.3 Deposit Operation

→ Customer enters the amount to deposit
→ Fraud check is performed on the amount
→ If amount exceeds Rs.50,000 a fraud warning is displayed
→ Customer confirms the transaction
→ BankingService.deposit() is called
→ Account balance is updated in the database
→ A DEPOSIT transaction record is created
→ Updated balance is displayed on the dashboard

5.2.4 Withdrawal Operation

→ Customer enters the amount to withdraw
→ Fraud check is performed
→ If amount exceeds Rs.10,000 an OTP is generated

→ Customer enters OTP from backend console
→ OTP is verified by MfaService
→ Sufficient balance check is performed
→ Account balance is deducted
→ A WITHDRAW transaction record is created
→ Updated balance is displayed

5.2.5 Fund Transfer Operation

→ Customer enters recipient account number and amount
→ Fraud detection check is performed
→ If amount exceeds Rs.10,000 OTP verification required
→ Sender account balance is verified
→ Recipient account is located by account number
→ Amount is deducted from sender account
→ Amount is credited to recipient account
→ TRANSFER transaction records created for both accounts
→ Success message displayed with updated balance

5.2.6 QR Code Payment

→ Customer clicks Show QR Code button
→ QRCodeSVG component generates a unique QR code
→ QR code encodes the account number and customer name
→ Other users can scan this QR code to send payments
→ QR code displays account number and customer name

5.2.7 Transaction History

→ All transactions are stored with timestamp, type, amount, description
→ Last 10 transactions are displayed on the dashboard
→ Each transaction shows type, amount, and timestamp using TransactionCard component

5.3 Module 3 - AI Fraud Detection Engine

5.3.1 Overview

The fraud detection module is one of the most technically sophisticated components of FinGenie AI. It uses the Strategy Design Pattern to implement a flexible and extensible fraud detection system that can switch between different detection algorithms based on the transaction context.

5.3.2 Strategy Pattern Implementation

The Strategy Pattern allows the fraud detection algorithm to be selected and changed at runtime without modifying the core fraud service logic.

❖ **FraudDetectionStrategy Interface:**

This interface defines the contract for all fraud detection algorithms. Any new fraud detection strategy can be added by simply implementing this interface without changing any existing code.

- BasicFraudStrategy:

→ This strategy implements basic rule-based fraud detection:
→ Checks if transaction amount exceeds defined thresholds
→ Monitors transaction frequency patterns
→ Flags transactions above Rs.50,000 as potentially suspicious
→ Generates basic risk scores based on amount analysis

- AdvancedFraudStrategy:

→ This strategy implements more sophisticated fraud detection:
→ Analyzes transaction patterns over time
→ Detects unusual spending behavior
→ Identifies transactions at unusual hours
→ Calculates comprehensive risk scores
→ Cross-references with historical transaction data

5.3.3 Fraud Alert System

→ Every transaction triggers the fraud detection check
→ FraudService selects the appropriate strategy
→ Strategy analyzes the transaction
→ If suspicious, a FraudAlert record is created
→ Alert is stored in the database with risk level and description
→ Admin can view all fraud alerts in the Admin Dashboard
→ Customer can view their personal fraud alerts in the Fraud Alerts page

5.4 Module 4 - AI Investment Advisor

5.4.1 Overview

The investment advisory module provides personalized investment recommendations based on the user's financial profile and risk appetite. It helps customers make informed investment decisions.

5.4.2 Features

❖ Risk Profiling:

→ Users are categorized into three risk profiles:
→ Low Risk - suitable for conservative investors preferring capital preservation
→ Medium Risk - balanced approach between growth and safety

→ High Risk - aggressive growth-oriented investment strategy

Mutual Fund Suggestions:

- Based on risk profile the system recommends appropriate mutual funds with expected returns and investment horizon.
- Portfolio Analytics:
- Users can view their complete investment portfolio including total invested amount, expected returns, and portfolio distribution across different asset classes.
- Savings Recommendation Engine:
- The system analyzes spending patterns and recommends optimal savings amounts based on income and expenses.

5.5 Module 5 - Smart Loan Management

5.5.1 Overview

The loan management module provides a complete loan application and management system with AI-based approval prediction to help customers understand their loan eligibility before applying.

5.5.2 Loan Application Process

- Customer fills loan application form with loan amount, tenure, annual income, credit score, and employment type
- System performs credit score analysis
- AI prediction engine evaluates loan eligibility
- Loan application is saved with PENDING status
- Prediction result shown to customer
- Admin can approve or reject applications

5.5.3 EMI Calculator

The EMI calculator helps customers understand their monthly payment obligations:

EMI Formula: EMI = P x R x (1+R)^N divided by ((1+R)^N minus 1)

Where:

→ P is the Principal loan amount
→ R is the Monthly interest rate (Annual rate divided by 12 divided by 100)
→ N is the Loan tenure in months

5.5.4 Credit Score Analysis

→ The system analyzes the following factors for credit score assessment:
→ Annual income relative to loan amount

→ Employment stability based on employment type
→ Loan to income ratio
→ Previous loan history

5.6 Module 6 - AI Chat Banking Assistant

5.6.1 Overview

The AI chatbot provides instant customer support for banking queries, account information, and financial guidance.

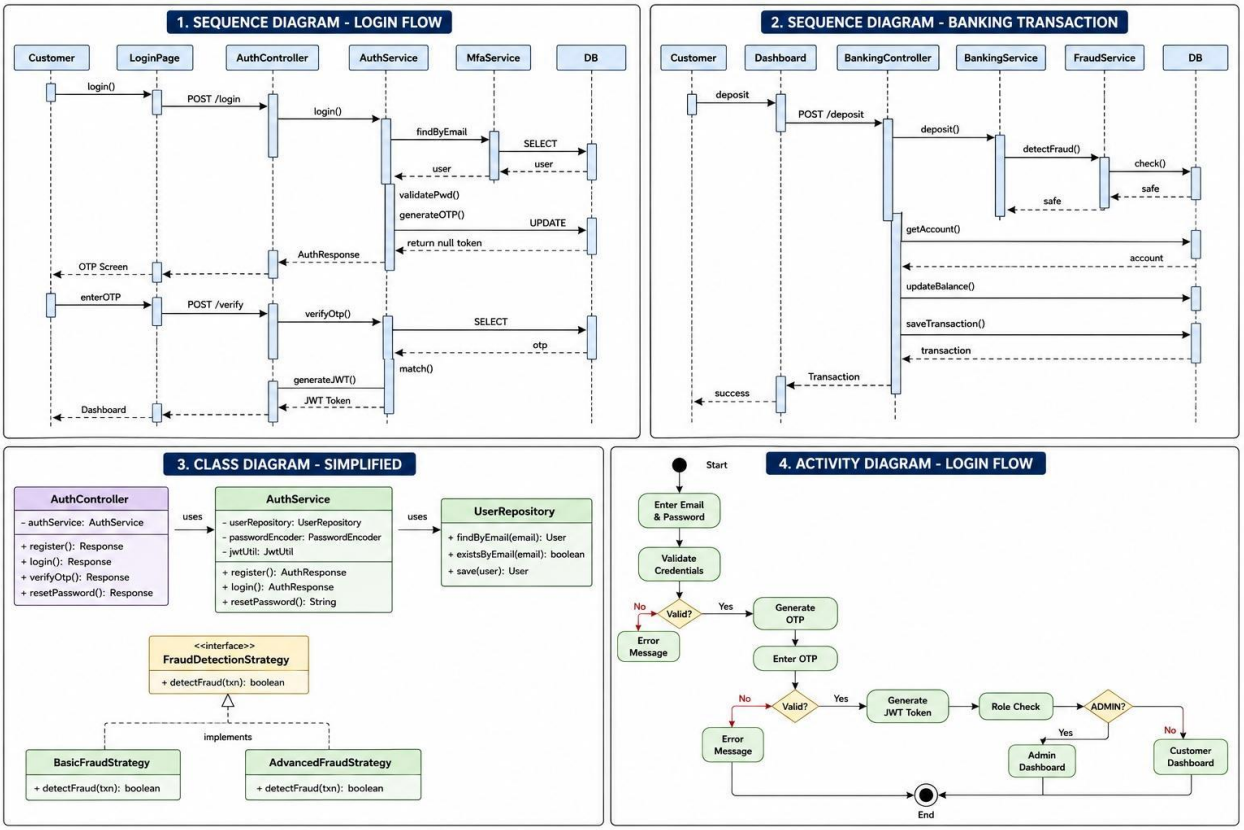
5.6.2 Features

→ Banking FAQ answering for common queries
→ Account balance and transaction queries
→ Loan and investment information
→ Smart escalation for complex queries
→ 24/7 availability for customer support

CHAPTER 6: DESIGN PATTERNS

6.1 Pattern 1 - Strategy Pattern

DML DIAGRAM



6.1.1 Definition

The Strategy Pattern is a behavioral design pattern that defines a family of algorithms, encapsulates each one, and makes them interchangeable. The pattern lets the algorithm vary independently from **clients that use it.**

6.1.2 Implementation in FinGenie AI

Location: Fraud Detection Module

Components:

→ FraudDetectionStrategy - Interface defining detectFraud() method
→ BasicFraudStrategy - Concrete strategy for basic fraud detection
→ AdvancedFraudStrategy - Concrete strategy for advanced fraud detection
→ FraudService - Context class that uses the strategy

6.1.3 Benefits

→ Open Closed Principle - open for extension, closed for modification
→ New fraud detection algorithms can be added without changing existing code
→ Algorithms can be switched at runtime based on transaction type

→ Each strategy can be tested independently

6.2 Pattern 2 - Factory Pattern

6.2.1 Definition

The Factory Pattern is a creational design pattern that provides an interface for creating objects in a superclass but allows subclasses to alter the type of objects that will be created.

6.2.2 Implementation in FinGenie AI

Location: Transaction Module

Components:

→ TransactionFactory - Factory class
→ createTransaction(type, amount, description) - Factory method
→ Returns appropriate Transaction object based on type
→ Transaction Types Created:
→ DEPOSIT transaction with positive amount
→ WITHDRAW transaction with negative amount
→ TRANSFER transaction with recipient information

6.2.3 Benefits

→ Single Responsibility Principle - object creation logic centralized
→ Reduces code duplication across deposit, withdraw, transfer operations
→ Easy to add new transaction types
→ Improves maintainability and readability

6.3 Pattern 3 - Singleton Pattern

6.3.1 Definition

The Singleton Pattern ensures a class has only one instance and provides a global point of access to it.

6.3.2 Implementation in FinGenie AI

→ Location: SecurityConfig, JwtUtil, Service classes
→ Spring Boot's @Bean and @Component annotations implement the Singleton pattern automatically. The IoC container ensures only one instance of each bean is created and shared across the application.
→ Singleton Beans in FinGenie AI:
→ SecurityConfig - single security configuration instance
→ JwtUtil - single JWT utility instance

→ All Service classes - single instance per service

→ All Repository classes - single instance per repository

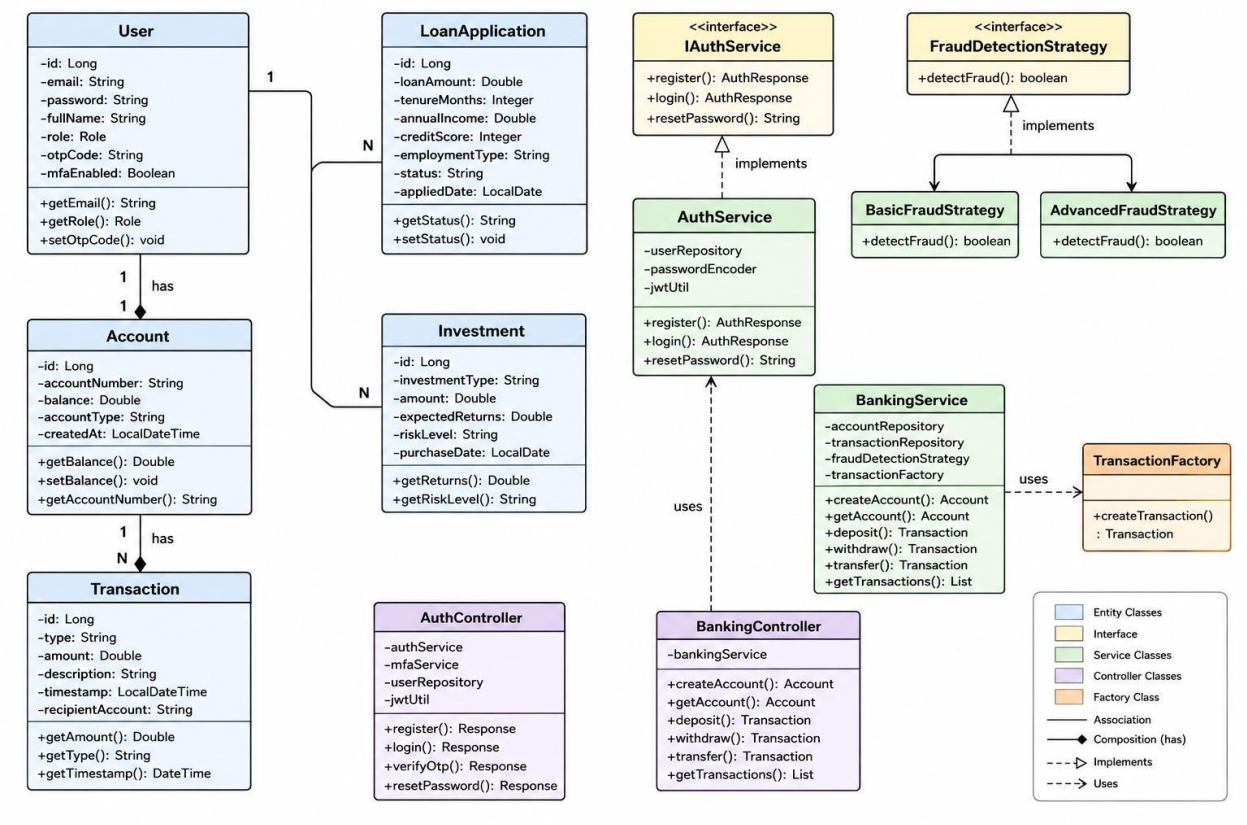
6.3.3 Benefits

→ Memory efficient - only one instance created

→ Consistent state across the application

→ Thread-safe instance management by Spring container

CLASS DIAGRAM



CHAPTER 7: SECURITY IMPLEMENTATION

7.1 Spring Security Configuration

FinGenie AI implements comprehensive security through Spring Security with the following configuration:

❖ **CSRF Protection:**

CSRF is disabled as the application uses stateless JWT authentication. JWT tokens provide sufficient protection against cross-site request forgery attacks.

- ❖ Session Management:

Session creation policy is set to STATELESS meaning no server-side sessions are created. All authentication state is maintained through JWT tokens on the client side.

- ❖ CORS Configuration:

Cross-Origin Resource Sharing is configured to allow requests from the React frontend running on localhost:3005 and localhost:3000.

- ❖ Endpoint Authorization:

Public endpoints - /api/auth/login, /api/auth/register, /api/auth/verify-otp are accessible without authentication

All other endpoints require valid JWT token in the Authorization header

7.2 JWT Authentication Filter

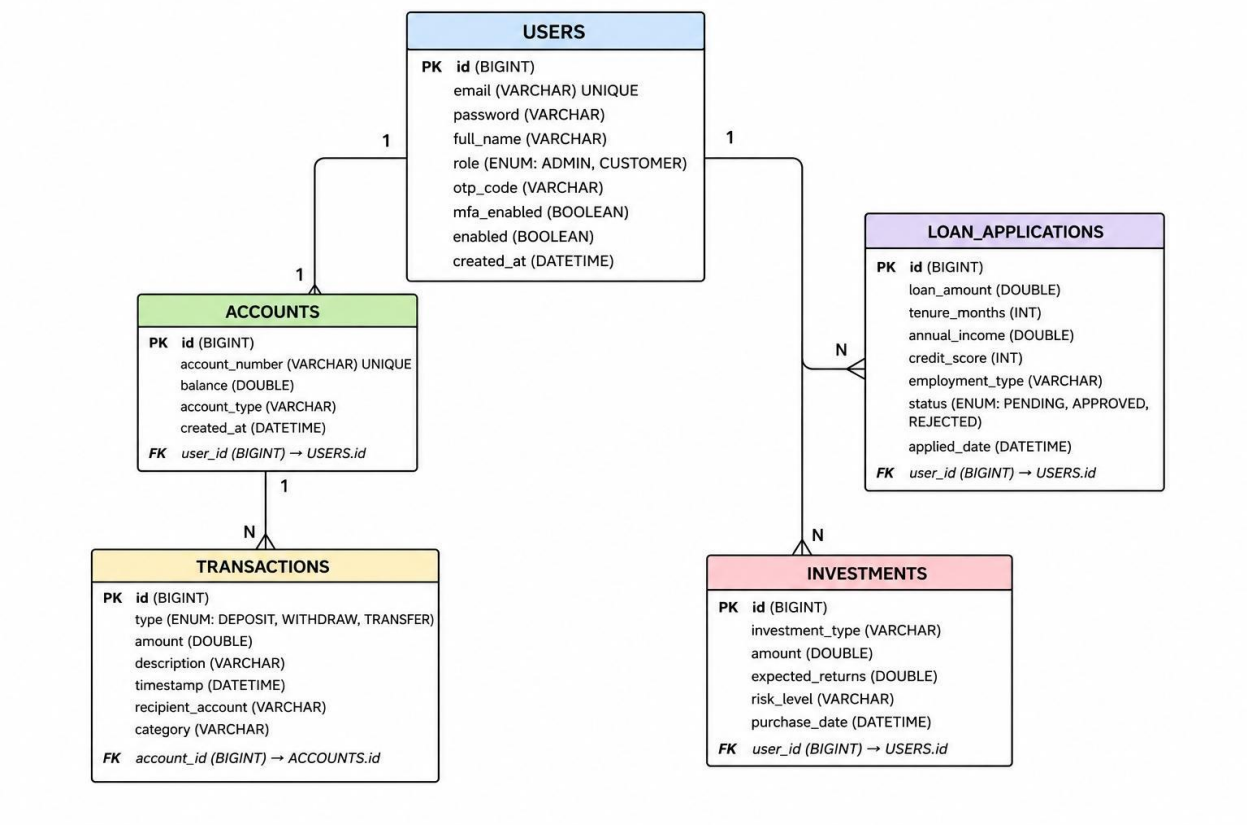
→ JwtAuthFilter extends OncePerRequestFilter and intercepts every HTTP request:
→ Extracts Authorization header from the request
→ Validates the Bearer token format
→ Extracts JWT token from the header
→ Validates token using JwtUtil
→ Extracts user email and role from token claims
→ Creates UsernamePasswordAuthenticationToken
→ Sets authentication in SecurityContextHolder
→ Continues filter chain

7.3 Password Security

All user passwords are encrypted using BCryptPasswordEncoder with a strength factor of 10. BCrypt is a adaptive hashing function designed for password hashing that incorporates a salt to protect against rainbow table attacks.

CHAPTER 8: DATABASE DESIGN

ER DIAGRAM



8.1 Entity Relationship Description

USER to ACCOUNT - One to One:

Each user has exactly one bank account. The account table has a foreign key user_id referencing the users table.

ACCOUNT to TRANSACTION - One to Many:

One account can have many transactions. The transaction table has a foreign key account_id referencing the accounts table.

USER to LOAN_APPLICATION - One to Many:

One user can have multiple loan applications. The loan_applications table has a foreign key user_id referencing the users table.

USER to INVESTMENT - One to Many:

One user can have multiple investments. The investments table has a foreign key user_id referencing the users table.

8.2 Database Tables

- users table:

Stores all user information including encrypted passwords, roles, and OTP codes for two-factor authentication.

- accounts table:

Stores bank account information with unique account numbers prefixed with FG and current balance.

- transactions table:

Stores complete transaction history with type, amount, description, timestamp, and recipient account for transfers.

- loan_applications table:

Stores all loan application details with approval status, credit score, income information, and employment type.

- investments table:

Stores investment records with type, amount, expected returns, risk level, and purchase date.

CHAPTER 9: API DOCUMENTATION

9.1 Authentication APIs

- POST /api/auth/register

Description: Register a new user account

Request Body: email, password, fullName

Response: AuthResponse with token, email, role, fullName

Status: 200 OK on success, 400 Bad Request if email exists

- POST /api/auth/login

Description: Login with email and password, triggers OTP generation

Request Body: email, password

Response: AuthResponse with null token, email, role, fullName

Status: 200 OK on success, 401 Unauthorized on invalid credentials

- POST /api/auth/verify-otp

Description: Verify OTP and get JWT token

Request Body: email, otp

Response: AuthResponse with JWT token, email, role, fullName

Status: 200 OK on success, 401 Unauthorized on invalid OTP

- POST /api/auth/reset-password

Description: Reset user password

Request Body: email, newPassword

Response: Success message string

Status: 200 OK on success, 404 if user not found

9.2 Banking APIs

- GET /api/banking/account

Description: Get account details for a user

Parameters: email as request parameter

Response: Account object with account number and balance

Authorization: JWT token required

- POST /api/banking/deposit

Description: Deposit money into account

Request Body: email, amount

Response: Transaction object

Authorization: JWT token required

- POST /api/banking/withdraw

Description: Withdraw money from account

Request Body: email, amount

Response: Transaction object

Authorization: JWT token required

- POST /api/banking/transfer

Description: Transfer money to another account

Request Body: email, toAccountNumber, amount

Response: Transaction object

Authorization: JWT token required

- GET /api/banking/transactions

Description: Get transaction history for a user

Parameters: email as request parameter

Response: List of Transaction objects

Authorization: JWT token required

CHAPTER 10: UNIT TESTING

10.1 Testing Approach

FinGenie AI implements unit testing for the service layer using JUnit 5 and Mockito framework. Unit tests verify individual business logic components in isolation by mocking dependencies.

10.2 Test Classes

- AuthServiceTest:

Tests the authentication business logic:

testRegister_Success - verifies successful user registration with correct response

testRegister_EmailAlreadyExists - verifies BadRequestException thrown for duplicate email

testLogin_Success - verifies successful login returns AuthResponse with null token

testLogin_WrongPassword - verifies UnauthorizedException thrown for wrong password

10.3 Mocking Strategy

Mockito is used to mock all external dependencies:

UserRepository is mocked to simulate database operations

PasswordEncoder is mocked to simulate password hashing

JwtUtil is mocked to simulate token generation

This ensures unit tests run without requiring a real database connection and execute extremely fast.

10.4 Test Execution

Tests are executed using Maven:

mvn test

CHAPTER 11: EXCEPTION HANDLING

11.1 Global Exception Handling

FinGenie AI implements centralized exception handling using @ControllerAdvice annotation on the GlobalExceptionHandler class. This ensures consistent error responses across all API endpoints.

11.2 Custom Exceptions

- BadRequestException (400):

Thrown when the request contains invalid data such as duplicate email during registration.

- ResourceNotFoundException (404):

Thrown when a requested resource such as user or account cannot be found in the database.

- UnauthorizedException (401):

Thrown when authentication fails such as invalid password or invalid OTP.

11.3 Error Response Format

All errors return a consistent JSON response format:

```
{
    "timestamp": "2026-06-15T10:30:00",
    "status": 400,
    "error": "Bad Request",
    "message": "Email already exists!",
    "path": "/api/auth/register"
}
```

CHAPTER 12: CONCLUSION

12.1 Project Summary

FinGenie AI successfully demonstrates a production-ready full-stack banking application that combines modern web technologies with intelligent features. The project covers all aspects of enterprise application development including security, database design, API development, frontend development, and testing.

12.2 Achievements

The following key achievements were accomplished in this project:

- **Technical Achievements:**

→ Successfully implemented a secure JWT plus OTP two-factor authentication system
→ Developed a complete banking operations module with fraud detection
→ Implemented three software design patterns - Strategy, Factory, and Singleton
→ Built a responsive and intuitive React frontend with dark and light mode support
→ Created a comprehensive admin analytics dashboard
→ Implemented global exception handling for consistent error responses
→ Written unit tests for service layer using JUnit 5 and Mockito

- **Functional Achievements:**

→ Complete user authentication with role-based access control
→ Full banking operations - deposit, withdrawal, fund transfer
→ Real-time fraud detection with alert system
→ Investment portfolio management and advisory
→ Loan application with EMI calculator
→ AI chatbot for customer support
→ QR code payment generation

12.3 Learning Outcomes

→ Through the development of FinGenie AI, the following learning outcomes were achieved:
→ Practical understanding of Spring Boot microservices architecture
→ Implementation of Spring Security with JWT authentication
→ Application of software design patterns in real-world scenarios
→ Full-stack development experience with React and Spring Boot integration
→ Database design and ORM with Hibernate and JPA
→ RESTful API design and development best practices
→ Unit testing with JUnit and Mockito

12.4 Future Enhancements

→ The following enhancements can be made to FinGenie AI in future versions:
→ Email-based OTP delivery instead of console printing
→ Real ML model for loan prediction and fraud detection
→ Push notifications for transaction alerts

→ Mobile application using React Native
→ Microservices architecture for better scalability
→ Kubernetes deployment for container orchestration
→ Real-time transaction monitoring using WebSockets
→ "FinGenie AI - Banking Made Intelligent"
→ This document was prepared as part of the Java Full Stack with React Capstone Project submission.